

TU DRESDEN · ZIH · DIGITIZATION & DATA
ANALYTICS

Machine Learning II Supervised Learning, Classification & Model Evaluation

Complete Study Guide · Visual Notes · Worked Examples ·
Exam-Style Q&A

Based on: Lecture 5 — "Digitization and Data Analytics" (Jana Klimpke, TU
Dresden / CIDS ZIH)

Topics: Training & Learning · Decision Trees (ID3, Entropy, Information Gain) · Naïve
Bayes Classifier · Model Evaluation (Confusion Matrix, Precision/Recall, ROC, Cross-
Validation)

Prepared for: MSc Computer Science exam preparation

Table of Contents

1. Big-Picture Overview of the Lecture Series
2. Supervised Learning — Training and Learning
 1. Features, Labels, and the Learning Task
 2. Data Splitting: Train / Validation / Test
 3. Generalization, Overfitting & Underfitting
 4. Cross-Validation
3. Classification I — Decision Trees
 1. Concept and Structure
 2. ID3 Top-Down Induction
 3. Entropy
 4. Information Gain
 5. Fully Worked Example
 6. Random Forests
4. Classification II — Bayesian Methods
 1. Bayes' Theorem Basics
 2. Maximum a Posteriori Classification
 3. Naïve Bayes Classifier (with full worked example)
 4. Zero-Probability Problem & Laplace Correction
 5. Advantages / Disadvantages
5. Model Evaluation and Selection
 1. Confusion Matrix
 2. Accuracy, Error Rate, Sensitivity, Specificity
 3. Precision, Recall, F-Measure
 4. Holdout, Cross-Validation, Bootstrap
 5. ROC Curves & Model Comparison
6. Exam-Style Question Bank (40+ Q&A, TU Dresden style)
7. Formula Cheat-Sheet (1 page, quick revision)

1. Big-Picture Overview of the Lecture Series

This lecture ("Machine Learning II — Supervised Learning") is the fifth lecture in the module. Understanding where it sits in the overall course helps you connect concepts across exam questions.

#	Lecture	Relation to this lecture
1	Introduction	Course context
2	Data Preprocessing	Cleans the data that becomes X here
3	Research Data Management	Governance of datasets
4	Machine Learning I — Unsupervised Learning	No labels; clustering, distances (previous lecture)
5	Machine Learning II — Supervised Learning	← this guide
6	Machine Learning III	More classification/regression methods
7	Data Analytics & Statistics	Statistical testing background
8–9	Deep Learning I & II	Neural-network based supervised learning
10–11	Distributed Computing I & II	Scaling ML to big data
12	Consequences	Ethics / societal impact

Exam framing tip: Lecturers often ask you to contrast *unsupervised* (Lecture 4: clustering, no labels, "what groups exist?") with *supervised* learning (Lecture 5: labeled data, "predict y from x "). Be ready to state the difference in one sentence: **supervised learning uses a training set of (input, output) pairs to learn a mapping $f: x \rightarrow y$; unsupervised learning only has inputs x and must discover structure.**

Structure of This Specific Lecture

1. **Supervised Learning** – Training and Learning (data splitting, overfitting, generalization)
2. **Classification**
 - 2.1 Decision Trees
 - 2.2 Bayes Classification Methods
3. **Model Evaluation and Selection** (accuracy, confusion matrix, ROC, cross-validation)

2. Supervised Learning — Training and Learning

2.1 Features, Labels, and the Learning Task

Supervised learning (a.k.a. **predictive analysis**) is the task of learning a mapping $\mathbf{f} : \mathbf{\bar{x}} \rightarrow \mathbf{\bar{y}}$ from a **training set** of labeled input-output pairs:

$$D = \{ (\bar{\mathbf{x}}_j, \bar{\mathbf{y}}_j) \}_{j=1}^N \text{ — } N \text{ training examples}$$

- $\mathbf{\bar{x}}$ — feature vector (attributes / covariates / predictors), e.g. x_1 = petal length
- $\mathbf{\bar{y}}$ — output vector (label / response / target), e.g. species name
- **Goal:** learn f so that unseen inputs can be mapped to correct outputs

Depending on the type of y , the task is named differently:

Type of y	Task name	Example
Categorical (discrete classes)	Classification	spam vs. non-spam, Iris species
Real number	Regression	house price, temperature forecast
Complex structured object	Structured prediction	parse trees, proof trees

Probabilistic view: Rather than a deterministic function, supervised learning can be framed as fitting a **conditional model** $p(\mathbf{y} | \mathbf{x})$ — a distribution over outputs given inputs. This is the view underlying the Naïve Bayes classifier discussed later.

Worked micro-example: spam filtering

Each email is represented as a binary feature vector over a dictionary of d words (e.g. $d \approx 100,000$): $\bar{\mathbf{x}} = (\text{insurance, learning, the, dating, medicament, ...}) \in \{0,1\}^d$, and the label is $y \in \{0,1\}$ where 1 = spam. The goal is to estimate $f(\mathbf{x})$ such that $y = f(\mathbf{x})$.

Types of training data you may be asked about

Data type	Typical use
Real-valued vectors	Flowers, patients, weather variables
Text	Named-entity recognition, sentiment, argument mining
Sequences	DNA / tumor classification
Images	Object & shape recognition
Video	Gesture recognition
Time series	Forecasting

2.2 Data Splitting: Train / Validation / Test

Labeled data (X, Y) is split into three **non-overlapping, independent** subsets:

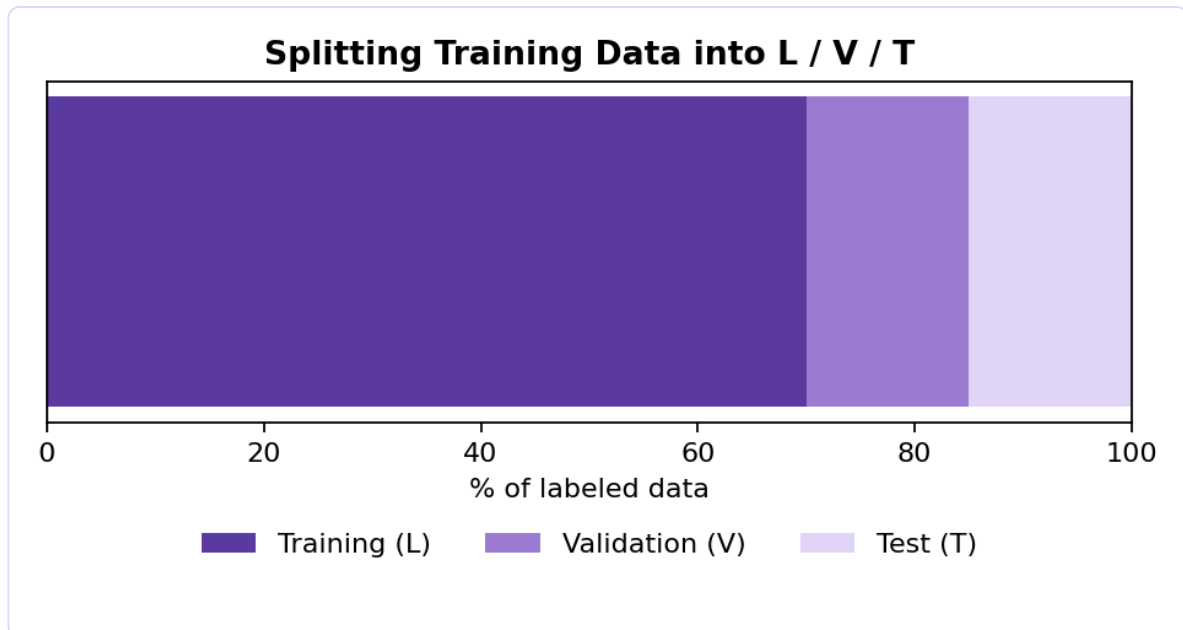


Figure 2.1 — Typical proportions for splitting labeled data into Learning, Validation, and Test sets.

Set	Symbol	Purpose
Training / Learning set	$L = (X_L, Y_L)$	"Anything goes": hunches, preliminary testing, trying many models, removing outliers
Validation set	$V = (X_V, Y_V)$	Model <i>selection</i> — comparing competing models by predictive ability
Test set	$T = (X_T, Y_T)$	Final, unbiased performance assessment of the <i>fully specified</i> model

Golden rule (guaranteed exam question): "Never test your model on the same data it was trained on!" Doing so gives an overly optimistic (biased) accuracy estimate because the model may have memorized the training data rather than learned generalizable patterns.

2.3 Generalization, Overfitting & Underfitting

Task: given training data (X, Y) , build a model $f(X)$ that predicts $y \in Y_{\text{new}}$ for previously unseen $x \in X_{\text{new}}$.

Strategy: estimate f on (X, Y) and *hope* it also works on new data — this hope is called **generalization**.

Overfitting: $f(x)$ predicts the training labels $y \in Y$ almost perfectly, but fails to predict $y' \in Y_{\text{new}}$ — the model has memorized noise/idiosyncrasies of the training set instead of the underlying pattern.

Underfitting: the model is too simple to capture the pattern even in the training data (high training error already).

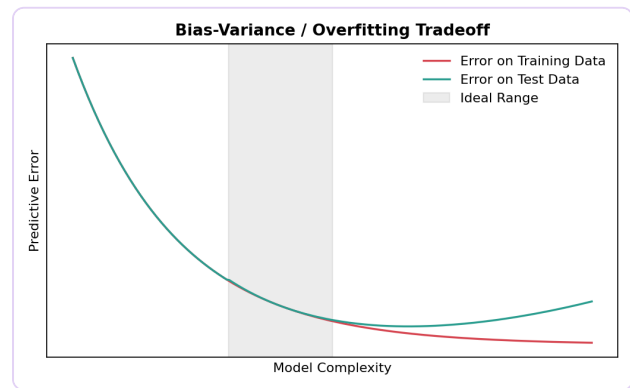


Figure 2.2 — Classic U-shaped test-error curve: as model complexity grows, training error keeps falling but test error eventually rises again (overfitting).

Diagnosing errors (a favorite flow-chart question)

Symptom	Diagnosis
Train error big	Underfitting
Train error small, validation error big	Overfitting
Train & validation small, test error big	Train/test distributions differ (data leakage / non-representative split)
All errors small	Good model

2.4 Cross-Validation

Splitting once into train/validation is "brutal" (wasteful and high-variance). The refinement is **k-fold cross-validation** ($k = 10$ is most popular):

1. Randomly partition data into k mutually exclusive, roughly equal-sized folds.
2. At iteration i , use fold D_i as the validation/test fold and the remaining $k-1$ folds as training data.
3. Repeat for all k folds; report the **average performance**.

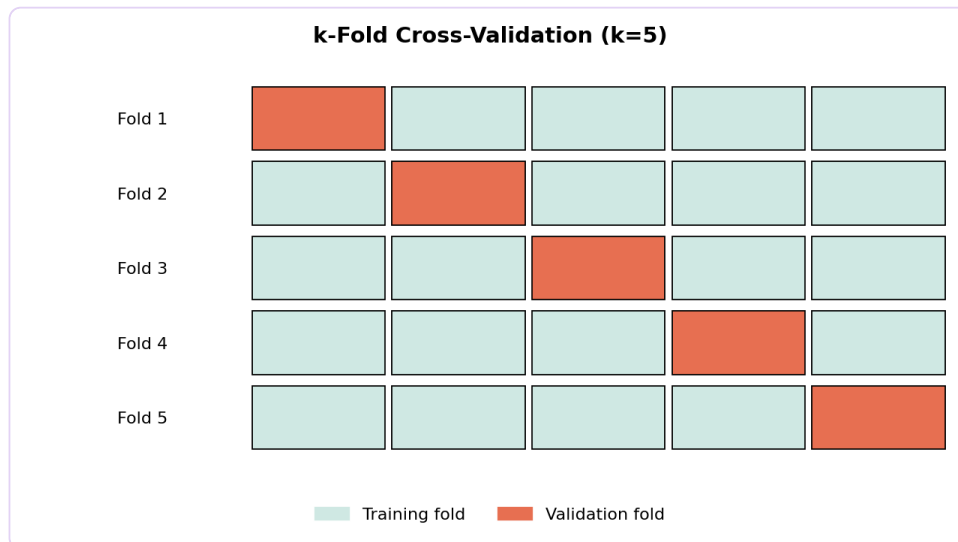


Figure 2.3 — 5-fold cross-validation: in each round a different fold (red) is held out for validation.

Variant	Description
Leave-one-out CV	k = number of tuples (used for very small datasets)
Stratified CV	folds preserve the overall class distribution → relatively low bias and variance
Holdout method	one random split (e.g. 2/3 train, 1/3 test)
Random subsampling	repeat holdout k times, average the accuracies
.632 Bootstrap	sample d tuples with replacement from a dataset of size d ; $\approx 63.2\%$ end up in the training bootstrap, $\approx 36.8\%$ form the test set (works well for small datasets)

$$Acc(M) = (1/k) \sum_{i=1..k} [0.632 \times Acc(M)_i_{test} + 0.368 \times Acc(M)_i_{train}]$$

Section 2 Takeaways

- Supervised learning = learn $f: \vec{x} \rightarrow \vec{y}$ from labeled data D .
- Split data into Training / Validation / Test — never evaluate on training data.

- Overfitting = great on train, bad on unseen data; underfitting = bad everywhere.
- Cross-validation (esp. stratified k-fold) gives a more robust accuracy estimate than a single holdout split.

3. Classification I — Decision Trees

3.1 Concept and Structure

A decision tree encodes classification rules as a tree:

- **Inner node:** a test on one feature
- **Branch:** one outcome/value of that feature's test
- **Leaf:** a class label (the classification decision)

Classic worked dataset ("Play Sport?"):

Outlook	Temperature	Humidity	Windy	Sport
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

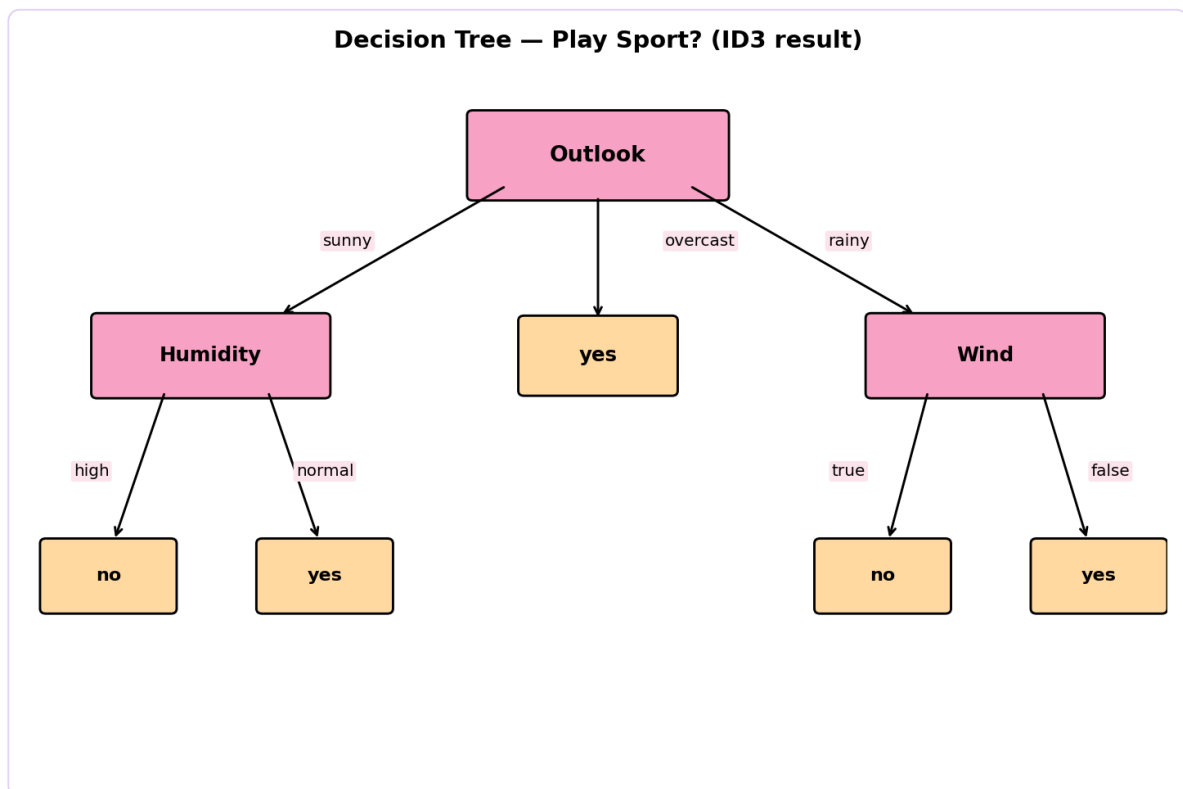


Figure 3.1 — Resulting decision tree: Outlook is the root; Humidity and Wind refine the "sunny" and "rainy" branches respectively.

Two competing goals when building a tree: (1) unknown data should be correctly classified, and (2) the tree should be small (short paths → good generalization, easy interpretation, Occam's razor).

3.2 ID3 Top-Down Induction

The classic algorithm (Quinlan, 1986) grows the tree greedily from the root:

1. A := the **"best" feature** for the decision at this node (measured by Information Gain).
2. Use A as the decision test at this node.
3. For each value of A, create a new child (successor) node.
4. Sort the training tuples into these child nodes according to their value of A.
5. If the training data at a child node is **perfectly classified** (pure) → STOP (make it a leaf); otherwise, recurse on that child using the remaining features.

3.3 Entropy — Measuring Impurity

Entropy $H(X)$ measures the **uncertainty / impurity** of a variable X . It is the expected number of bits needed to encode a randomly drawn member of X under an optimal (shortest-length) code.

$$H(X) = - \sum_{i=1..n} p(x_i) \cdot \log_2(p(x_i))$$

For a binary class ($\oplus/\ominus$) with proportions $p_+ = p/(p+n)$ and $p_- = n/(p+n)$:

$$\text{Entropy}(X) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

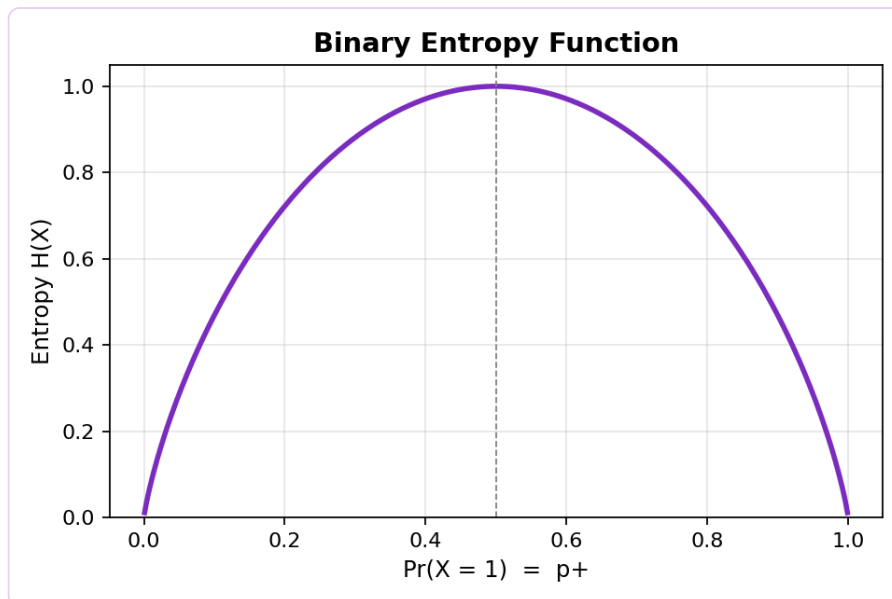


Figure 3.2 — Binary entropy peaks at 1 bit when classes are perfectly balanced ($p=0.5$) and is 0 when the set is pure ($p=0$ or $p=1$).

Example set	Computation	H(X)
4 items, proportions $\frac{1}{4}, \frac{1}{4}, \frac{1}{2}$ (3 classes)	$-(\frac{1}{4}\log_2 \frac{1}{4} + \frac{1}{4}\log_2 \frac{1}{4} + \frac{1}{2}\log_2 \frac{1}{2}) = -(\frac{1}{4} \cdot -2 + \frac{1}{4} \cdot -2 + \frac{1}{2} \cdot -1)$	1.5
4 identical items (pure)	$-(1 \cdot \log_2 1)$	0

Intuition: High entropy = high disorder / mixed classes = "hard to predict." Zero entropy = a pure node = "already know the answer." A good split should *reduce* entropy the most.

3.4 Information Gain

Information Gain $IG(X, A)$ measures the expected reduction in entropy from a prior state to a state where attribute A has been observed (i.e., how much a split on A improves purity):

$$IG(X, A) = H(X) - \sum_{v \in \text{values}(A)} (|X_v| / |X|) \cdot H(X_v)$$

The "best" feature at each node is the one with **maximum Information Gain**.

Toy comparison: which split is best?

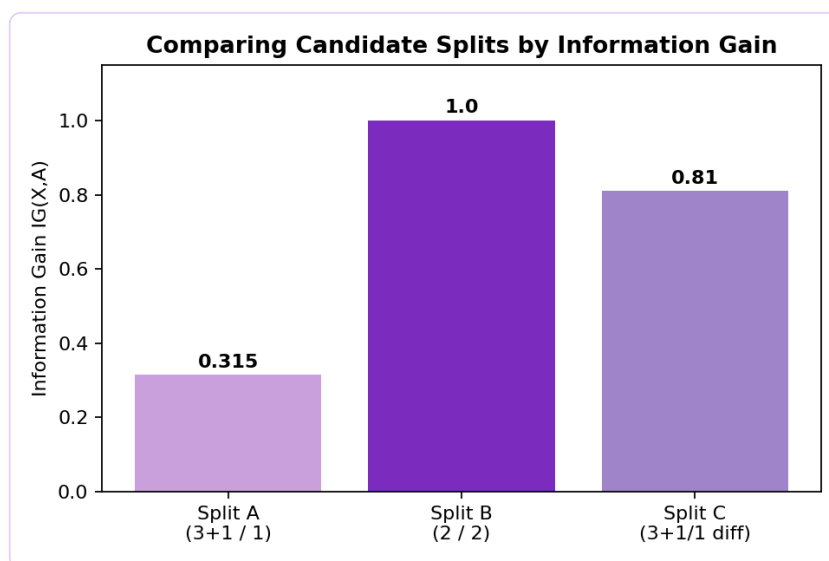


Figure 3.3 — Given $H(X)=1.5$, three candidate splits yield $IG = 0.315$, 1.0 , and 0.81 respectively. Split B ($IG=1.0$, perfectly pure children) wins.

3.5 Fully Worked Example — Sports Dataset

Root entropy over 14 examples (9 "yes", 5 "no"):

$$H(X) = - (9/14)\log_2(9/14) - (5/14)\log_2(5/14) \approx 0.94$$

Candidate split on Outlook (sunny: [2+,3−], overcast: [4+,0−], rainy: [3+,2−]):

$$IG(X, Outlook) = 0.94 - (5/14) \cdot H(\text{sunny}) - (4/14) \cdot H(\text{overcast}) - (5/14) \cdot H(\text{rainy}) \approx \mathbf{0.246}$$

Candidate split on Windy (true: [3+,3−], false: [6+,2−]):

$$IG(X, Windy) = 0.94 - (6/14) \cdot H(\text{true}) - (8/14) \cdot H(\text{false}) \approx \mathbf{0.048}$$

Since $0.246 > 0.048$ (and Outlook also beats Temperature and Humidity in the full computation), **Outlook is chosen as the root**. The "overcast" branch is already pure ([4+, 0−] → leaf "yes"), while "sunny" and "rainy" are recursively split further (on Humidity and Wind respectively, as shown in Figure 3.1).

Common exam task: You will very likely be given a small table (≤ 16 rows) and asked to (a) compute the root entropy, (b) compute IG for 2–3 candidate attributes, and (c) state which attribute ID3 picks as the root, and possibly draw the resulting first split. Practice doing this by hand — no calculators are usually needed since $\log_2$ values for common fractions ($\frac{1}{2}$, $\frac{1}{3}$, $\frac{1}{4}$, etc.) are typically given.

3.6 Random Forest Algorithms (Extension)

A Random Forest is an **ensemble learning** method that:

- Trains many decision trees on different randomly bootstrapped subsets of the training data.
- Each tree is trained on a randomly selected subset of the *features* as well (feature bagging).
- Handles missing/noisy/irrelevant features more gracefully than a single tree.
- Is much less prone to overfitting than a single decision tree (variance reduction via averaging/voting).
- Can be trained in parallel (trees are independent given the bootstrap samples).

Section 3 Takeaways

- Decision trees recursively split data using the feature with highest Information Gain (ID3).
- Entropy quantifies impurity; Information Gain quantifies the entropy reduction from a split.
- Goal: small trees that still classify unseen data correctly (avoid overfitting to noise).
- Random Forests = many decorrelated trees (bagging + feature bagging) → lower variance, better generalization.

4. Classification II — Bayesian Methods

4.1 Bayes' Theorem — Basics

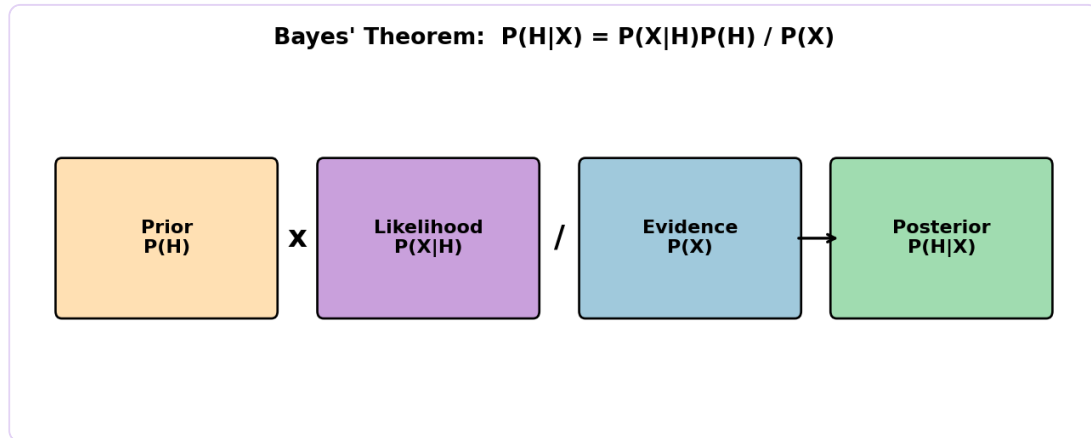


Figure 4.1 — The four components of Bayes' theorem.

$$P(H | X) = [P(X | H) \cdot P(H)] / P(X)$$

Term	Name	Meaning
$P(H X)$	Posterior	Probability that hypothesis H holds, given the observed attributes X (what we want)
$P(H)$	Prior	Initial probability of H , before seeing any evidence
$P(X H)$	Likelihood	Probability of observing X , assuming H is true
$P(X)$	Evidence	Probability of observing X at all (marginal, constant across classes)

Informally: **posterior = likelihood × prior / evidence.**

4.2 Maximum a Posteriori (MAP) Classification

Given m classes $C_1, \dots, C_m$ and a tuple $X = (x_1, \dots, x_n)$, classification means finding the class that **maximizes the posterior probability**:

$$P(C_i | X) = [P(X | C_i) \cdot P(C_i)] / P(X)$$

Since $P(X)$ is the same for every class, we only need to **maximize the numerator**:

$$\text{Predicted class} = \operatorname{argmax}_i P(X | C_i) \cdot P(C_i)$$

Practical difficulty: computing $P(X | C_i)$ directly requires knowing the joint distribution over all feature combinations — this needs huge amounts of data and is computationally expensive. This motivates the "naïve" simplifying assumption below.

4.3 Naïve Bayes Classifier

Simplifying (naïve) assumption: all features are **conditionally independent** given the class:

$$P(X | C_i) = \prod_{k=1..n} P(x_k | C_i) = P(x_1|C_i) \cdot P(x_2|C_i) \cdots P(x_n|C_i)$$

This drastically reduces the cost: we now only need to estimate per-attribute class-conditional probabilities, not the full joint distribution.

Attribute type	How $P(x_k C_i)$ is estimated
Categorical	# tuples in C_i with value x_k for A_k , divided by $ C_{i,D} $
Continuous	Gaussian density with mean μ and standard deviation σ estimated from C_i 's tuples: $P(x_k C_i) = 1/(\sqrt{2\pi}\sigma) \cdot e^{-(x-\mu)^2/2\sigma^2}$

Fully Worked Example — "Buys Computer?"

Classes: C_1 = buys_computer="yes", C_2 = buys_computer="no". New tuple to classify:

$$X = (\text{age} \leq 30, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit_rating} = \text{fair})$$

Step 1 — Priors (from 14 training tuples, 9 yes / 5 no):

$$P(\text{yes}) = 9/14 = 0.643 \quad P(\text{no}) = 5/14 = 0.357$$

Step 2 — Class-conditional likelihoods (counted from the training table):

Condition	Given "yes"	Given "no"
age ≤ 30	2/9 = 0.222	3/5 = 0.6
income = medium	4/9 = 0.444	2/5 = 0.4
student = yes	6/9 = 0.667	1/5 = 0.2
credit_rating = fair	6/9 = 0.667	2/5 = 0.4

Step 3 — Multiply likelihoods (naïve independence assumption):

$$P(X | \text{yes}) = 0.222 \times 0.444 \times 0.667 \times 0.667 = 0.044$$

$$P(X | \text{no}) = 0.600 \times 0.400 \times 0.200 \times 0.400 = 0.019$$

Step 4 — Multiply by prior (this is proportional to the posterior since $P(X)$ cancels):

$$P(X|\text{yes}) \cdot P(\text{yes}) = 0.044 \times 0.643 = \mathbf{0.028}$$

$$P(X|\text{no}) \cdot P(\text{no}) = 0.019 \times 0.357 = \mathbf{0.007}$$

Conclusion: Since $0.028 > 0.007$, the naïve Bayes classifier predicts **X belongs to class "buys_computer = yes."**

Exam checklist for this exact example type: (1) state the naïve independence assumption explicitly; (2) show the prior computation; (3) show each conditional probability with its fraction (do not skip steps); (4) multiply correctly (naïve Bayes = product, not sum!); (5) compare the two final scores and state the predicted class clearly.

4.4 The Zero-Probability Problem & Laplace Correction

If *any* single conditional probability $P(x_k|C_i)$ is zero (because that attribute value never appeared with that class in the training data), the *entire product* becomes zero — even if all other attributes strongly favor that class. This is clearly undesirable.

Example: dataset with 1000 tuples; income=low (0 tuples), income=medium (990), income=high (10).

Laplacian correction: add 1 to every count (and add the number of attribute values to every denominator):

$$\text{Prob}(\text{income}=\text{low}) = 1/1003 \quad \text{Prob}(\text{income}=\text{medium}) = 991/1003$$

$$\text{Prob}(\text{income}=\text{high}) = 11/1003$$

The corrected estimates stay close to the uncorrected ones but avoid the zero-probability trap.

4.5 Naïve Bayes — Advantages, Disadvantages, Extensions

Advantages	Disadvantages
Easy to implement	Independence assumption rarely holds exactly → some loss of accuracy
Good results in most practical cases	Real-world attributes are often correlated (e.g., patient age, family history, symptoms, and disease are all interdependent)

Incremental — each new training example can update probabilities; combines well with prior knowledge	Cannot model dependencies among attributes directly
Provides an optimal-decision baseline even when computationally intractable in general	—

Extension for dependencies: *Bayesian Belief Networks* generalize naïve Bayes by explicitly modeling a graph of conditional dependencies between attributes.

Other classification methods mentioned (know the names for breadth questions)

- Rule-Based Classification
- Support Vector Machines (SVM)
- k-Nearest-Neighbor Classifier (k-NN)
- Genetic Algorithms
- (Deep) Neural Networks

Section 4 Takeaways

- Naïve Bayes assumes **conditional independence of features given the class** — this is what makes it "naïve."
- MAP classification: pick the class maximizing $P(X|C_i) \cdot P(C_i)$ ($P(X)$ cancels out).
- Use Laplace (add-one) smoothing to avoid zero probabilities wiping out the whole product.
- Bayesian Belief Networks relax the independence assumption at the cost of more complex modeling.

5. Model Evaluation and Selection

Always evaluate on a held-out **validation/test set** of class-labeled tuples — never on the training set.

5.1 Confusion Matrix

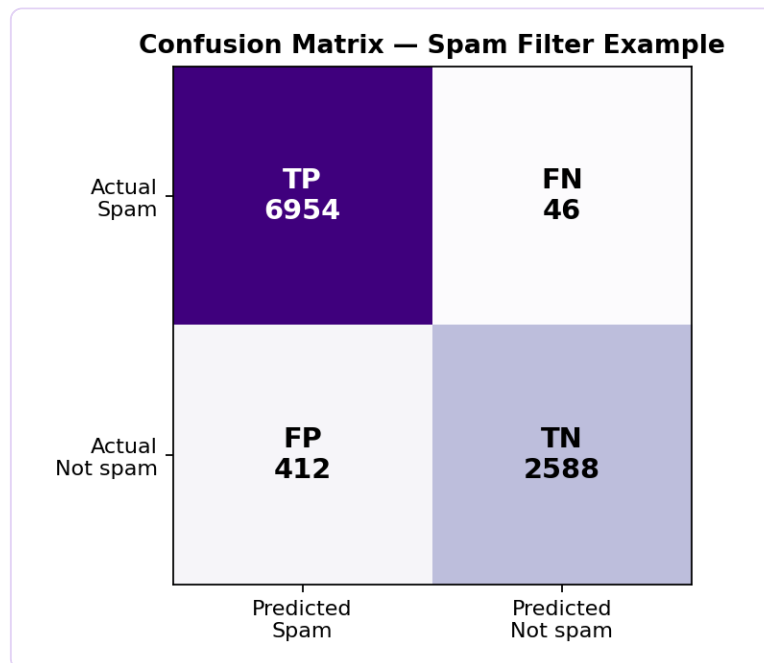


Figure 5.1 — Confusion matrix for a spam-filter example (10,000 emails total).

	Predicted C_1	Predicted $\neg C_1$
Actual C_1	True Positives (TP)	False Negatives (FN)
Actual $\neg C_1$	False Positives (FP)	True Negatives (TN)

Entry C_{ij} = number of tuples of actual class i that the classifier labeled as class j .

5.2 Accuracy, Error Rate, Sensitivity, Specificity

$$\text{Accuracy} = (TP + TN) / \text{All} \quad \text{Error rate} = (FP + FN) / \text{All} = 1 - \text{Accuracy}$$

$$\text{Sensitivity (TPR / Recall)} = TP / P \quad \text{Specificity (TNR)} = TN / N$$

Class imbalance problem: when one class is rare (e.g. fraud, cancer detection), accuracy can be misleadingly high (a classifier that always predicts "no fraud" can be

99% accurate but useless). This is why **sensitivity** (catching the rare positive class) and other metrics below matter more in imbalanced settings.

5.3 Precision, Recall, and F-Measures

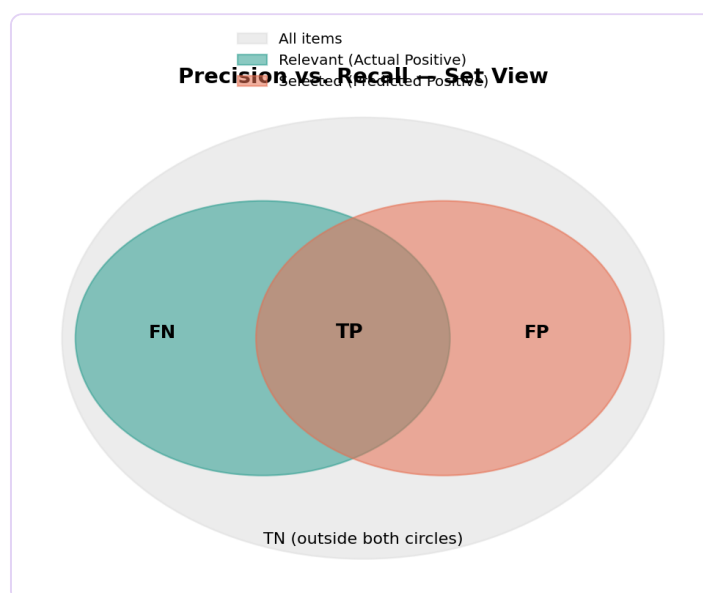


Figure 5.2 — Precision = "of what I selected, how much is actually relevant?" Recall = "of what's actually relevant, how much did I find?"

$$\text{Precision} = TP / (TP + FP) \quad \text{Recall} = TP / (TP + FN)$$

- **Precision** (exactness): what % of tuples labeled positive are actually positive.
- **Recall** (completeness): what % of actual positives did the classifier find.
- There is often an **inverse relationship** between precision and recall (raising one tends to lower the other).

$$F_{\beta} = [(1 + \beta^2) \cdot \text{precision} \cdot \text{recall}] / [\beta^2 \cdot \text{precision} + \text{recall}] \quad F_1 = (2 \cdot \text{precision} \cdot \text{recall}) / (\text{precision} + \text{recall})$$

F_{β} weighs recall β times as much as precision; F_1 (harmonic mean) weighs them equally.

Worked example (Spam filter, 10,000 emails; 7000 actual spam, 3000 actual non-spam)

	Predicted spam	Predicted not spam	Total	Recognition %
Actual spam	6954	46	7000	99.34% (sensitivity)
Actual not spam	412	2588	3000	86.26% (specificity)
Total	7366	2634	10000	95.42% (accuracy)

$$\text{Precision} = 6954/7366 = 94.41\% \quad \text{Recall} = 6954/7000 = 99.34\%$$

Exam tip: You may be given raw grids of predicted-vs-actual dots/icons (as in some slide decks) instead of a numeric table — practice counting TP/FP/FN/TN visually and then plugging into the formulas.

5.4 Holdout, Cross-Validation, and Bootstrap (recap with evaluation lens)

Method	Best for	Key idea
Holdout	Large datasets	Single random split, e.g., 2/3 train, 1/3 test
Random subsampling	Reducing variance of holdout	Repeat holdout k times, average accuracy
k-fold Cross-validation	General purpose, most common (k=10)	Every tuple used exactly once for testing
Leave-one-out	Very small datasets	k = number of tuples
Stratified CV	Imbalanced classes	Preserve class ratio in every fold
.632 Bootstrap	Small datasets	Sample with replacement; ~63.2%/36.8% train/test split

5.5 Comparing Classifiers: Statistical Significance

Mean error rates from cross-validation are just **estimates** of the true error on the population of future data. If classifier M_1 has a lower mean error rate than M_2 , this might just be due to chance. Use a **test of statistical significance** (e.g., a paired t-Test on the fold-wise error rates) — being mindful of that test's preconditions — before concluding that one model is genuinely better.

5.6 ROC Curves

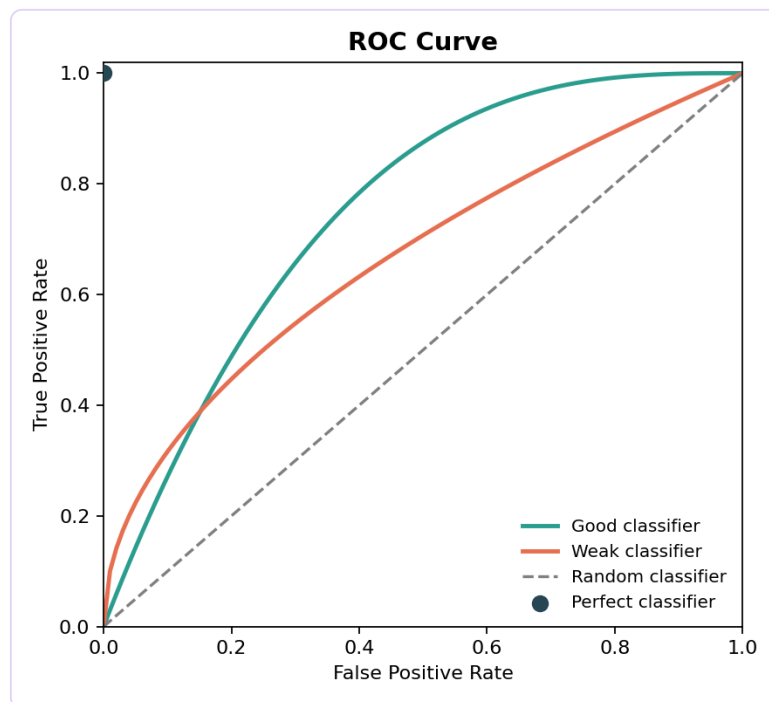


Figure 5.3 — ROC space: curves closer to the top-left corner (higher AUC) indicate better classifiers.

- **Origin:** signal detection theory.
- **Axes:** vertical = True Positive Rate (Sensitivity); horizontal = False Positive Rate ($1 - \text{Specificity}$).
- **Construction:** the model returns a probability/score per tuple; rank tuples in decreasing order of predicted positive-class probability and sweep a threshold from top to bottom, plotting (FPR, TPR) at each cut.
- **Diagonal line** = random guessing baseline.
- **Area Under the Curve (AUC):** a single-number summary of accuracy.
 - $\text{AUC} = 1.0 \rightarrow$ perfect classifier
 - $\text{AUC} = 0.5 \rightarrow$ no better than random (on the diagonal)
 - Closer to the diagonal ($\text{AUC} \rightarrow 0.5$) = less accurate model

5.7 Issues Affecting Model Selection (beyond accuracy)

Criterion	What it measures
Accuracy	How well the model predicts the class label
Speed	Training time + prediction/classification time
Robustness	Handling of noise and missing values
Scalability	Efficiency on very large / disk-resident data
Interpretability	How much insight/understanding the model provides (e.g., decision trees >> deep nets)

Other	Rule compactness, decision tree size, goodness of rules
-------	---

Section 5 Takeaways

- Confusion matrix → derive Accuracy, Error rate, Sensitivity/Recall, Specificity, Precision, F-measure.
- Under class imbalance, accuracy is misleading — prefer sensitivity/precision/recall/F1/AUC.
- Use k-fold (stratified) cross-validation for robust accuracy estimates; use statistical tests before declaring one model "better."
- ROC/AUC compares classifiers across all thresholds, not just one operating point.

6. Exam-Style Question Bank

Curated in the style of TU Dresden MSc Computer Science exams / oral exam prep. Answers are concise but exam-complete. Questions marked **hard** require synthesis across sections; **exam** marks a very frequently asked pattern.

A. Supervised Learning Fundamentals

Q: Define supervised learning formally, including the roles of D , N , $\vec{x}$, $\vec{y}$.

A: Supervised learning learns a function $f: \vec{x} \rightarrow \vec{y}$ from a training set $D = \{(\vec{x}_j, \vec{y}_j)\}_{j=1..N}$ of N labeled input-output pairs, where $\vec{x}$ is the feature vector (binary/categorical/real-valued attributes) and $\vec{y}$ is the label/class/output vector. The goal is that f generalizes to correctly predict y for new, unseen x .

Q: What distinguishes classification from regression?

exam

A: Both are supervised learning tasks that estimate $f: \vec{x} \rightarrow \vec{y}$. In classification, y is categorical (a discrete class label, e.g., "spam"/"non-spam"). In regression, y is a real number (e.g., predicted price).

Q: Why must the test set be independent of the training set?

A: If the model is evaluated on data it was trained on, it can appear to perform very well simply by having memorized that data (overfitting), giving an overly optimistic and non-generalizable accuracy estimate. The test set must reflect performance on genuinely unseen data.

Q: What is the difference between the validation set and the test set?

exam

A: The validation set is used *during* model development for model **selection** — comparing competing models/hyperparameters. The test set is used *only once*, after the final model is fully specified, to obtain an unbiased estimate of real-world performance. Using the test set for tuning would leak information and bias the final estimate.

Q: Define overfitting and underfitting, and explain how you would detect each from training/validation/test error alone.

A: Overfitting: low training error but high validation/test error (model fit noise/ idiosyncrasies of training data). Underfitting: high error even on the training set (model too simple to capture the pattern). Detect via the error-diagnosis flow: big train error $\rightarrow$ underfitting; small train error + big validation error $\rightarrow$ overfitting; small train & validation but big test error $\rightarrow$ train/test distributions differ.

Q: Explain k-fold cross-validation and why stratification matters. exam

A: The data is randomly split into k roughly equal folds. In each of k rounds, one fold is held out for validation/testing and the model is trained on the remaining $k-1$ folds; the reported accuracy is the average over all k rounds. Stratified CV ensures each fold preserves the overall class distribution of the full dataset, which reduces bias and variance in the estimate, especially for imbalanced datasets.

Q: When would you prefer leave-one-out CV or the .632 bootstrap over standard 10-fold CV? hard

A: Both are preferred for **small datasets**, where 10-fold CV would leave too little data per fold to train a reliable model, or where you want to use nearly all data for training in each round. Leave-one-out sets $k = \text{number of tuples}$ (maximal training data per round, but computationally expensive and high variance across rounds for small $k-1$ differences). The .632 bootstrap resamples with replacement, using $\sim 63.2\%$ of the data as an (in-bag) training set and the $\sim 36.8\%$ out-of-bag tuples as the test set, repeated many times and combined with a weighted formula ($0.632 \cdot \text{test-acc} + 0.368 \cdot \text{train-acc}$) to reduce pessimistic bias.

B. Decision Trees

Q: Describe the ID3 algorithm in your own words.

A: ID3 grows a decision tree top-down and greedily. At each node, it (1) selects the attribute A with the highest Information Gain as the decision test, (2) creates one child node per possible value of A , and (3) sorts the training tuples accordingly. If a child's tuples are already pure (all same class), it becomes a leaf; otherwise the process recurses on that child using the remaining attributes.

Q: Give the formula for entropy and explain, in one sentence, what it measures. exam

A: $H(X) = -\sum_i p(x_i) \cdot \log_2(p(x_i))$. It measures the uncertainty/impurity of a set X — equivalently, the expected number of bits needed to encode the class of a randomly drawn member under the optimal code. $H=0$ means pure (no uncertainty); $H=1$ (for two equally likely classes) means maximal uncertainty.

Q: Compute the entropy of a set with 9 positive and 5 negative examples.

A: $H(X) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) \approx 0.94$ bits.

Q: Give the Information Gain formula and explain how it is used to pick the splitting attribute. exam

A: $IG(X,A) = H(X) - \sum_{v \in \text{values}(A)} (|X_v|/|X|) \cdot H(X_v)$ — the entropy of the parent minus the weighted average entropy of the children produced by splitting on A. ID3 computes IG for every candidate attribute at a node and picks the attribute with the **maximum** IG as the test for that node, because it yields the greatest reduction in uncertainty.

Q: In the Play-Sport example, $IG(\text{Outlook})=0.246$ and $IG(\text{Windy})=0.048$. Which attribute does ID3 choose as the root, and why?

A: Outlook, because $0.246 > 0.048$ — splitting on Outlook reduces entropy the most (and, in the full computation, also beats Temperature and Humidity), so it is selected as the root of the tree.

Q: Why might a decision tree with 100% training accuracy be a bad model?

hard

A: A tree grown until every leaf is perfectly pure can end up encoding noise specific to the training set (deep, highly specific paths for individual outliers) rather than a general pattern — this is classic overfitting. It typically performs worse on unseen (validation/test) data than a shallower, pruned tree, even though its training accuracy looks perfect. This is why goals for tree induction explicitly include keeping the tree small, and why techniques like pruning or ensemble methods (Random Forests) are used in practice.

Q: What is a Random Forest and what two sources of randomness does it use? exam

A: A Random Forest is an ensemble of many decision trees. Randomness comes from (1) bagging — each tree is trained on a different bootstrap sample of the training tuples, and (2) feature subsampling — each tree (or each split) considers only a random subset of the available features. The forest's prediction is typically a majority vote (classification) or average (regression) over all trees, which reduces variance and overfitting compared with a single tree, and trees can be trained in parallel.

C. Naïve Bayes

Q: State Bayes' Theorem and identify each term (prior, likelihood, evidence, posterior). exam

A: $P(H|X) = P(X|H) \cdot P(H) / P(X)$. $P(H)$ is the prior (probability of H before evidence), $P(X|H)$ is the likelihood (probability of the evidence given H), $P(X)$ is the evidence (marginal probability of X across all hypotheses), and $P(H|X)$ is the posterior (updated probability of H after observing X).

Q: Why can we ignore $P(X)$ when doing MAP classification across several classes?

A: $P(X)$ is the same constant for every class C_i being compared (it does not depend on i), so it does not affect which class has the maximum posterior. We can therefore just maximize the numerator $P(X|C_i) \cdot P(C_i)$ instead of the full fraction.

Q: What is the "naïve" assumption in Naïve Bayes, and what practical problem does it solve? exam

A: The naïve assumption is **conditional independence of all features given the class**: $P(X|C_i) = \prod_k P(x_k|C_i)$. This avoids having to estimate a full joint distribution over all attribute combinations (which would need exponentially more data / computation), by instead only requiring per-attribute, per-class conditional probabilities.

Q: How is $P(x_k|C_i)$ estimated for (a) a categorical attribute and (b) a continuous attribute?

A: (a) Categorical: the fraction of tuples in class C_i that have value x_k for attribute A_k , i.e., $\text{count} / |C_{i,D}|$. (b) Continuous: usually modeled with a Gaussian density using the class-specific mean μ and standard deviation σ : $P(x_k|C_i) = (1/\sqrt{2\pi\sigma^2}) \cdot e^{-(x-\mu)^2/2\sigma^2}$.

Q: Given $P(\text{yes})=0.643$, $P(\text{no})=0.357$, and likelihoods 0.222, 0.444, 0.667, 0.667 (yes) vs. 0.6, 0.4, 0.2, 0.4 (no), classify X. Show your work. exam

A: $P(X|\text{yes}) = 0.222 \times 0.444 \times 0.667 \times 0.667 \approx 0.044$; $P(X|\text{no}) = 0.6 \times 0.4 \times 0.2 \times 0.4 \approx 0.019$. Then $P(X|\text{yes})P(\text{yes}) \approx 0.044 \times 0.643 \approx 0.028$ and $P(X|\text{no})P(\text{no}) \approx 0.019 \times 0.357 \approx 0.007$. Since $0.028 > 0.007$, classify X as "yes."

Q: What is the zero-probability problem and how does Laplace (add-one) smoothing fix it? exam

A: If any single attribute value never co-occurred with a class in the training data, its estimated conditional probability is 0, which zeroes out the entire product $P(X|C_i)$ regardless of how strongly other attributes support that class. Laplace correction adds 1 to every count (and adds the number of possible values of that attribute to the denominator), ensuring every probability is strictly positive while staying close to the uncorrected estimates when counts are large.

Q: List two advantages and two disadvantages of Naïve Bayes, and name an extension that addresses one disadvantage.

A: Advantages: easy to implement; good performance in many real-world tasks despite the naïve assumption; incremental updating; provides a useful baseline.

Disadvantages: the conditional-independence assumption is usually false in practice (e.g., symptoms and diseases are correlated), causing some loss of accuracy; it cannot directly model attribute dependencies. Extension: Bayesian Belief Networks explicitly model a graph of dependencies between attributes.

Q: Compare Naïve Bayes and Decision Trees on interpretability, handling of continuous attributes, and sensitivity to irrelevant features. **hard**

A: Decision trees are highly interpretable (explicit if-then rules) and handle both categorical and continuous attributes naturally via threshold splits (e.g., $PL < 2.5?$), but can overfit on irrelevant features unless pruned. Naïve Bayes is also fairly interpretable (probabilities per feature), handles continuous attributes via a Gaussian assumption (which may be inaccurate if the true distribution isn't Gaussian), and is comparatively robust to irrelevant features since each feature only contributes a multiplicative factor rather than being used to split the data — but it can be hurt if features are strongly correlated, violating the independence assumption.

D. Model Evaluation

Q: Define TP, FP, TN, FN using the spam-filter example.

A: TP: actual spam correctly predicted as spam. FP: actual non-spam incorrectly predicted as spam. TN: actual non-spam correctly predicted as non-spam. FN: actual spam incorrectly predicted as non-spam.

Q: Given TP=6954, FP=412, FN=46, TN=2588, compute accuracy, precision, recall, and specificity. **exam**

A: Accuracy = $(6954 + 2588) / 10000 = 95.42\%$. Precision = $6954 / (6954 + 412) = 94.41\%$. Recall/Sensitivity = $6954 / (6954 + 46) = 99.34\%$. Specificity = $2588 / (2588 + 412) = 86.27\%$.

Q: Why is accuracy a poor metric for class-imbalanced problems? Give a concrete counter-example. **exam**

A: If 99% of tuples belong to the negative class (e.g., "not fraud"), a trivial classifier that always predicts "not fraud" achieves 99% accuracy while having 0% recall for the (important) positive class. Accuracy hides this complete failure to detect the rare class, so metrics like sensitivity, precision, recall, F-measure, or AUC are preferred.

Q: Explain the trade-off between precision and recall, and describe when you'd prioritize one over the other. hard

A: Increasing the decision threshold for "positive" typically raises precision (fewer false positives) but lowers recall (more false negatives), and vice versa — hence the usual inverse relationship. Prioritize recall when missing a positive is very costly (e.g., cancer screening, fraud detection — false negatives are dangerous). Prioritize precision when false positives are very costly or disruptive (e.g., spam filters that must not block important emails, legal document flagging with high review cost per false alarm).

Q: What does the F1 score represent, and why use the harmonic mean rather than the arithmetic mean of precision and recall?

A: $F1 = 2 \cdot \text{precision} \cdot \text{recall} / (\text{precision} + \text{recall})$ is the harmonic mean of precision and recall — a single number balancing both. The harmonic mean is used (rather than the arithmetic mean) because it penalizes extreme imbalance between precision and recall much more strongly: if either value is very low, F1 will be low even if the other is very high, which better reflects that a "0.99 / 0.01" classifier is not actually a good classifier on average.

Q: Describe how an ROC curve is constructed and what AUC = 0.5 means.

exam

A: The classifier outputs a probability/score per tuple; ranking tuples by this score and sweeping a decision threshold from the most-likely-positive tuple downward produces a sequence of (FPR, TPR) points, which are plotted to form the ROC curve. AUC = 0.5 corresponds to the diagonal line, i.e., a model performing no better than random guessing; AUC = 1.0 is a perfect classifier that separates all positives and negatives.

Q: Why should mean error rates from two classifiers be compared using a statistical test rather than simply comparing the numbers? hard

A: Mean error rates estimated from a finite sample (e.g., via k-fold CV) are noisy estimates of the true population error, subject to sampling variability. A small numeric difference between two classifiers' mean errors could easily arise by chance rather than reflecting a genuine difference in performance. A statistical significance test (e.g., a paired t-test across folds, respecting its assumptions such as approximate normality of the per-fold differences) is needed to determine whether the observed difference is unlikely to be due to chance alone.

Q: Besides accuracy, what other criteria should you weigh when selecting a model for production use? exam

A: Speed (training time and prediction/classification time), robustness (to noise and missing values), scalability (to large / disk-resident datasets), and interpretability (how much insight/understanding the model provides — important for trust, debugging, and regulatory contexts), plus task-specific measures like decision-tree size or rule compactness.

E. Synthesis / Cross-Topic Questions hard

Q: Contrast how Decision Trees and Naïve Bayes each address the "curse of dimensionality" / combinatorial explosion of feature combinations.

A: Decision trees address it implicitly by only ever testing one attribute at a time along any root- to-leaf path, and by choosing the most informative attribute first (via Information Gain), so irrelevant or low-information attributes may never even be used. Naïve Bayes addresses it explicitly via the conditional independence assumption, which lets it estimate $P(X|C_i)$ as a product of n simple univariate probabilities instead of one exponentially large joint distribution over all attribute combinations.

Q: A colleague reports 99.9% training accuracy with a deep, unpruned decision tree but only 62% accuracy on a held-out test set. Diagnose the issue and propose two concrete remedies.

A: This is a textbook case of **overfitting** — the tree has grown deep enough to memorize idiosyncrasies (including noise) of the training set rather than the general Outlook/Humidity/Wind-style pattern. Remedies: (1) prune the tree (limit depth, minimum leaf size, or post-hoc pruning) to keep it smaller and force it to rely on the attributes with genuinely high Information Gain; (2) use cross-validation to select tree hyperparameters, or switch to an ensemble method like Random Forests, which averages many bootstrap-trained trees to reduce variance without sacrificing much predictive power.

Q: You must choose between a Naïve Bayes classifier and a Decision Tree for a medical diagnosis task with strongly correlated symptoms (e.g., fever and cough often co-occur) and a requirement that clinicians can audit every decision. Which would you pick, and what evaluation metric would you emphasize?

A: Since symptoms are strongly correlated, Naïve Bayes' conditional-independence assumption is violated, which can hurt its calibration/accuracy (though it may still rank cases reasonably well). A decision tree naturally captures some of these dependencies through sequential splits and offers clear, auditable if-then rules matching the interpretability requirement, so it (or an interpretable ensemble with feature importances) is often preferred here. Given the likely class imbalance in disease datasets and the high cost of missing a true case, emphasize **sensitivity/recall** (and possibly the ROC/AUC across thresholds) rather than raw accuracy.

Q: Explain, end to end, the full workflow you would use to build and evaluate a classifier from a raw labeled dataset, referencing every concept covered in this lecture. hard

A: (1) Split the labeled data into training/validation/test sets (or prepare for k-fold CV). (2) Train candidate models (e.g., a Decision Tree via ID3 using Information Gain, and a Naïve Bayes classifier with Laplace smoothing) on the training folds. (3) Use the validation set / cross-validation folds for model selection, monitoring for overfitting/underfitting via the train-vs-validation error gap. (4) Compute a confusion matrix on held-out data and derive accuracy, precision, recall, specificity, and F1 — paying special attention to class imbalance. (5) Compare candidate models using ROC/AUC and, if the numeric difference is small, a statistical significance test across folds. (6) Weigh non-accuracy criteria (speed, robustness, scalability, interpretability) before final selection. (7) Only at the very end, evaluate the chosen, fully specified model once on the untouched test set to report an unbiased final performance estimate.

7. Formula Cheat-Sheet (Quick Revision)

Entropy & Information Gain

$$H(X) = -\sum p(x_i) \cdot \log_2 p(x_i) \quad | \quad IG(X,A) = H(X) - \sum_v (|X_v|/|X|) \cdot H(X_v)$$

Bayes / Naïve Bayes

$$P(H|X) = P(X|H)P(H)/P(X) \quad | \quad \text{Predict} = \underset{j}{\operatorname{argmax}} P(X|C_j)P(C_j) \quad | \quad P(X|C_j) = \prod_k P(x_k|C_j)$$

Confusion-Matrix Metrics

Metric	Formula
Accuracy	$(TP+TN)/All$
Error rate	$(FP+FN)/All = 1 - \text{Accuracy}$
Sensitivity / Recall / TPR	$TP/(TP+FN) = TP/P$
Specificity / TNR	$TN/(TN+FP) = TN/N$
Precision	$TP/(TP+FP)$
F1	$2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$
Fβ	$(1+\beta^2) \cdot \text{Precision} \cdot \text{Recall} / (\beta^2 \cdot \text{Precision} + \text{Recall})$

Bootstrap

$$Acc(M) = (1/k) \sum_i [0.632 \cdot Acc(M_i)_{test} + 0.368 \cdot Acc(M_i)_{train}]$$

One-Page Mental Model

Stage	Key question	Key tool
Data prep	How do I split labeled data?	Train / Validation / Test, k-fold CV
Model fit	What structure best separates classes?	Decision Tree (IG) or Naïve Bayes (Bayes' rule)
Diagnose	Is my model over/underfitting?	Train vs. validation error gap

Evaluate	How good is it, really?	Confusion matrix → Accuracy/Precision/Recall/ F1/ROC
Select	Which model should I ship?	Statistical test + speed/robustness/ interpretability

Final exam-day reminder: practice deriving entropy and information gain by hand on a small table, practice the 4-step Naïve Bayes worked example until automatic, and always be ready to state *why* a metric was chosen — TU Dresden exams frequently reward justification, not just correct numbers.

— End of Study Guide — Compiled from "Digitization and Data Analytics: Machine Learning II" (TU Dresden, CIDS ZIH, Jana Klompke) for exam preparation. —